THE COMPLETE TUTORIAL · VERSION 0.1.0

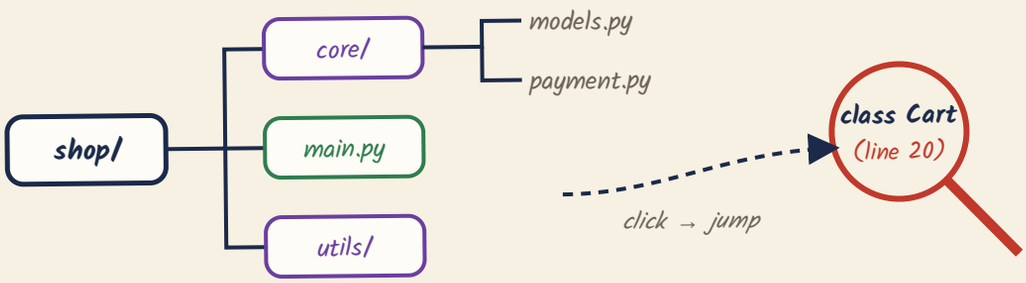
Fou

Developer-controlled codebase inspection and navigation for Python

From the first command to JSON pipelines, clickable output and publishing

Written by **Mohammad Mehdi Masoumi**

github.com/mohammadmehdimasoumi



Name: **Mohammad Mehdi Masoumi**

Fou v0.1.0

Contents

1. Introduction p.3
2. Installation and First Steps p.5
3. The Tutorial Project p.8
4. Core Concepts p.10
5. Configuring an Inspection p.13
6. The Nine Inspection Types p.17
7. Details Mode and Code-Line Counting p.24
8. Working with Results in Python p.28
9. Clickable Output p.31
10. JSON Output and Saving p.36
11. Command-Line Reference p.39
12. Errors and Robustness p.42
13. Performance and Caching p.44
14. Practical Recipes p.46
15. Architecture and Extending Fou p.50
16. Testing and Publishing to PyPI p.52
17. Limitations, FAQ and Troubleshooting p.57
A: Appendix: Cheat Sheet p.59
B: Appendix: Glossary p.60
About: About the Author p.61

How to use this notebook

Every topic goes *intuition → usage → why it works → real output*. Yellow = key term. Green-bar block = real output of the tutorial project. Dark block = terminal. Boxes: green = tip · red dashed = remember · purple = key note · pink = common mistake · green filled = summary. A code snippet looks like this:

```
project = inspect("./shop")
```

Notebook edition of the Fou guide by Mohammad Mehdi Masoumi · github.com/mohammadmehdimasoumi

---

happy inspecting! ✏️

1 What is Fou?

The question — *Every developer eventually asks: what is actually in this codebase? Which classes exist? Where is that helper defined? Which files import which modules? How big is each file, honestly, once blank lines and comments are ignored?*

You can answer with `grep`*, an IDE sidebar, or an afternoon of reading. Fou answers with one function call. You tell it exactly what you want to see, and it answers from the source code, the same way every time.*

> **The one-sentence summary**
>
> *Fou reads a project's source code with Python's own* `ast` *and* `tokenize` *modules and gives back structured, predictable results that you can* ***print, iterate over, save as JSON, or click****.*

Why another inspection tool?

Tools that explore code fall into two camps. ***IDEs*** *are powerful but live inside a window and are hard to script.* ***Linters and doc generators*** *are scriptable but built for one narrow purpose. Fou sits in the gap: a small library whose entire job is to describe a project's structure and let you decide what to do with that description.*



Fou fills the gap between the two camps

Fou IS	**Fou is NOT**
a ***deterministic*** *library and CLI*	*an AI agent — no model decides what is relevant*
a reader of source code (static)	*something that runs or imports your code*
offline, standard-library only	*something that uses the network*

> **Summary**
>
> *Fou = what exists inside this project, and what is its structure? — answered by a tool that is predictable, offline and fully under your control.*

1 Features, Scope and Audience

Six promises

Promise	What it means
Deterministic	Same project + same arguments = byte-identical output, even across machines.
Developer-controlled	You choose directory, depth, extensions, exclusions and types.
Zero dependencies	Only the Python standard library — installing Fou pulls nothing else in.
Two interfaces, one engine	Python API in a script, or the `fou` command in a terminal. Same code path.
Robust	A syntax error in one file is recorded; everything else is still inspected.
Navigable	Terminal hyperlinks open your editor at the exact line of the class or function you clicked.

What this release contains

Version 0.1.0 ships the `inspect` module. Planned future modules: `find`, `locate`, `run`, `relation`. The shared **core layer** (locations, elements, results, errors) already exists so they can be built on the same project model later.

What Fou will NOT do

- It **does not run** your code. Names made by `setattr`, `exec`, class factories or decorator magic are invisible.
- It calls no AI service and sends your code nowhere. It **only reads** — never modifies.
- It analyzes `.py` files only. Other files may appear in the tree, but nothing is extracted from them.

Audience: anyone who can read basic Python and use a terminal. Every output here comes from one tiny tutorial project.

2 Installing Fou

Goal — from zero to your first inspection in five minutes.

Requirements

- **Python 3.9+** (Fou uses only the standard library).
- A terminal on Linux, macOS or Windows. (Clickable links need a terminal that supports them — see chapter 9.)

Install

Once the package is published, installing is a single command:

```
$ pip install fou
Successfully installed fou-0.1.0
```

To work on Fou itself — or try the source before it is published — use **editable mode**. Python then imports your working copy, so every edit takes effect immediately:

```
$ pip install -e .
```

Want the test suite? Add the `dev` extra, which installs pytest:

```
$ pip install -e ".[dev]"
```

> **Use a virtual environment**
> Run `python -m venv .venv` and activate it first. Fou's default exclude list already skips folders named `.venv` and `venv`, so your environment never pollutes an inspection.

Checking the installation

Fou installs a console command called `fou`. You can also run it as a module — handy when the scripts folder is not on your PATH:

```
$ fou --version
fou 0.1.0
$ python -m fou --version
fou 0.1.0
```

Both go through the same `main()` function, so the result is identical.

2 Your First Inspection

The `inspect` subcommand takes the path of a project directory. With no other options it prints the *file tree* and, next to every Python file, its number of code lines (how they are counted: chapter 7).

```
$ fou inspect ./shop
Project: shop
├── __init__.py
│ -> 1 line
│
├── config.py
│ -> 4 lines
│
├── main.py
│ -> 7 lines
│
├── core/
│ ├── __init__.py
│ │ -> 0 lines
│ │
│ ├── models.py
│ │ -> 20 lines
│ │
│ └── payment.py
│ -> 7 lines
│
└── utils/
 └── helpers.py
 -> 8 lines
```

> **Sorting**
>
> Files are listed **before** directories, and both groups are sorted case-insensitively. That is why the top-level files come before `core/` and `utils/`.

Asking for something specific

The `-t` (type) option chooses what to inspect. Ask for classes:

```
$ fou inspect ./shop -t classes
Project: shop
Classes (2)
core/models.py
├── Product (line 9)
└── Cart (line 20)
```

Each class appears under its file, with the line where it starts.

2 Details Mode and the Python API

Add -d for details

*Details add docstrings, decorators, line counts and the **methods inside each class**:*

```
$ fou inspect ./shop -t classes -d
Project: shop
Classes (2)
core/models.py
├── Product (line 9)
│ -> 6 lines
│ -> decorators: @dataclass
│ -> doc: A product that can be sold.
│ └── with_tax(self) -> float (line 15)
└── Cart (line 20)
 -> 11 lines
 -> doc: A shopping cart.
 ├── ... (__init__, add)
 ├── total(self) -> float (line 30)
 │ -> decorators: @property
 │ -> property method
 └── empty_message() -> str (line 34)
 -> decorators: @staticmethod
 -> static method
```

The same thing from Python

*Everything the command line does is a library call. The entry point `inspect` returns a **Project**; each inspection method on it returns an **InspectionResult**.*

```
from fou import inspect

project = inspect("./shop")
result = project.classes()

print(result)             # human-readable text
print(len(result))        # how many classes were found
for cls in result:        # iterate over the elements
    print(cls.qualname, "->", cls.path, "line", cls.location.lineno)
```

```
Project: shop
Classes (2)
...
2
Product -> core/models.py line 9
Cart -> core/models.py line 20
```

Nothing is printed unless you print it — methods return result objects (safe inside programs, tests, web services). `result.show()` prints for you.

3 A Small Shop (1/2)

Why a tutorial project *— examples are easier when you can see the code being inspected. Every output in this notebook comes from a tiny project called* `shop`*: seven Python files that still contain every construct Fou understands — a dataclass, a regular class, a property, a static method, an async function, a decorator, nested functions, constants, variables, and internal + external imports.*

Layout: `__init__.py` · `config.py` · `main.py` · `core/` (`__init__`, `models`, `payment`) · `utils/helpers.py`

config.py

```
"""Application settings."""
from typing import Final

DEBUG = False
MAX_ITEMS: Final = 100
TAX_RATE = 0.2
```

core/models.py

```
"""Domain models."""
from dataclasses import dataclass
from typing import List
from ..config import TAX_RATE

@dataclass
class Product:
    """A product that can be sold."""
    name: str
    price: float

    def with_tax(self) -> float:
        """Return the price including tax."""
        return round(self.price * (1 + TAX_RATE), 2)

class Cart:
    """A shopping cart."""
    def __init__(self) -> None:
        self.items: List[Product] = []
    def add(self, product: Product) -> None:
        self.items.append(product)
    @property
    def total(self) -> float:
        return sum(p.with_tax() for p in self.items)
    @staticmethod
    def empty_message() -> str:
        return "Your cart is empty"
```

3 A Small Shop (2/2)

core/payment.py

```
"""Payment helpers."""
import asyncio
from .models import Cart

async def charge(cart: Cart) -> bool:
    await asyncio.sleep(0.1)
    return cart.total > 0

def refund(amount: float) -> str:
    return f"Refunded {amount}"
```

utils/helpers.py

```
"""Small helper functions."""
import functools

def slugify(text):
    return text.lower().replace(" ", "-")

def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

main.py

```
"""Entry point."""
from .core.models import Cart, Product
from .utils.helpers import log_calls

@log_calls
def run():
    cart = Cart()
    cart.add(Product("Book", 12.5))
    print(cart.total)
```

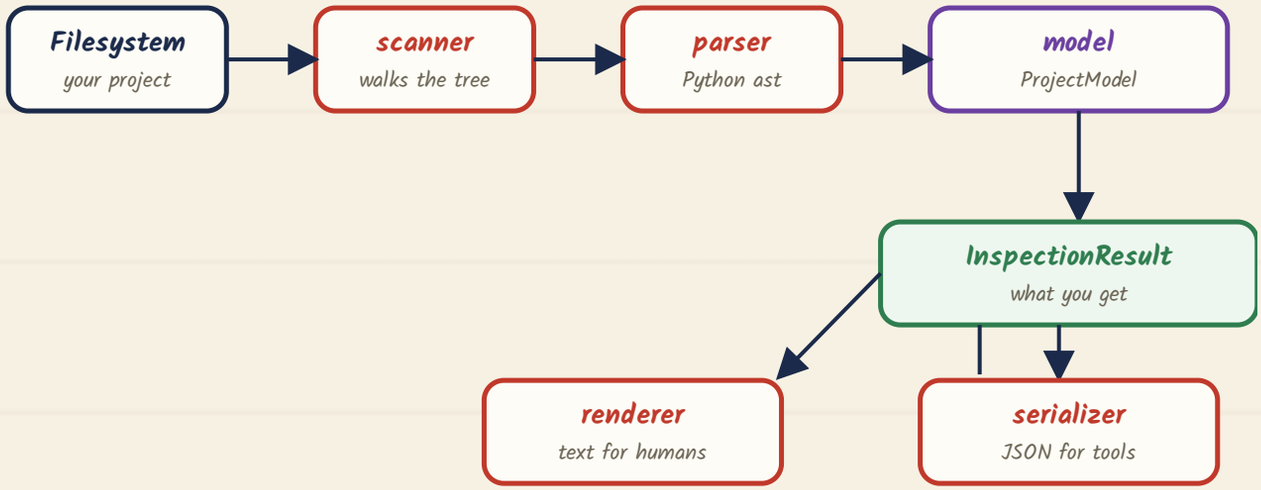
Also: `shop/__init__.py` *(docstring +* `__version__`*), an empty* `core/__init__.py`*, and a* `README.txt` *shown only for non-Python extensions.*

Why the folder is called shop

Fou derives module names from `__init__.py` *files, so classes get names like* `shop.core.models.Cart`.

4 The Pipeline

Mental model — *Fou keeps four concerns strictly separate: reading the filesystem, parsing code, representing what was found, and presenting it. Each stage only reads what the previous stage produced.*



the Fou pipeline — every stage only reads the previous stage's output

Stage	Job
scanner	Walks the directory following `depth`, `extensions`, `exclude`, `hidden`; produces a tree of file records. Opens files only when it has to.
parser	Parses each Python file with the standard `ast` module and extracts elements.
model	Internal, renderer-independent representation (`ProjectModel`, `FileModel`, `Element`). Knows nothing about printing or JSON.
InspectionResult	What every public method returns.
renderer / serializer	Turn a result into terminal text, or into JSON.

Why bother?

A new output format means writing a new serializer — not touching the parser. Separation of concerns is what lets future modules reuse the same project model.

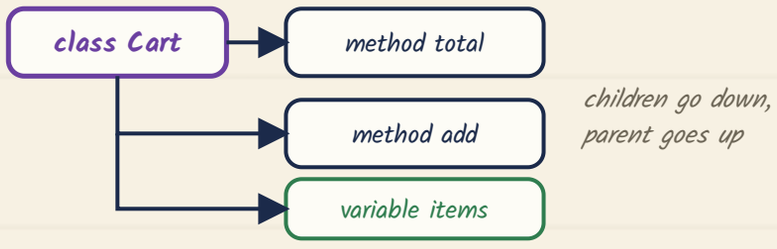
4 Objects and Element Kinds

The four objects you will meet

Object	What it is	How you get it
Project	A directory you chose to inspect; holds the cached scan.	`fou.inspect(location)`
InspectionResult	The answer to one question (classes, files, ...).	`project.classes()`, `project.query()`
Element	One discovered thing: a class, function, import, ...	iterate over a result
FileModel	One discovered file plus its line statistics.	`result.files`, or iterate `files()`

Seven element kinds of Element

kind	Meaning
`import`	One imported name (`from x import y` produces one element for `y`).
`function`	A `def` / `async def` not directly in a class body. Check `is_async`.
`method`	A `def` / `async def` directly inside a class body.
`class`	A `class` statement, nested classes included.
`variable`	A name binding with a scope: module, class, instance or local.
`constant`	A module/class-level UPPER_CASE name, or anything annotated `Final`.
`decorator`	A decorator, linked to the thing it decorates.



4 Locations, Ids and Laziness

Locations

Each element has a `location` with four numbers: `lineno`, `col`, `end_lineno`, `end_col`.

- **Lines are 1-based**, like your editor shows them.
- **Columns are 0-based** character offsets, like Python's `ast`. When building a clickable link, Fou converts the column to 1-based because editors expect that.

```
from fou import inspect

cart = inspect("./shop").classes()["classes"][1]
print(cart)
print(cart.location)
print(cart.module, "|", cart.qualname, "|", cart.id)
```

```
<Element class 'Cart' core/models.py:20>
core/models.py:20
shop.core.models | Cart | core/models.py::Cart
```

Relationships are kept through *stable string ids* like `core/models.py::Cart.total` (format: `path::qualname`).

Lazy by design

Calling `inspect(...)` is **instant**: it only validates arguments (so a typo in `depth` fails right away) and never touches the filesystem. The first inspection method triggers the scan, and after that everything is cached:

1. The filesystem is scanned **once**.
2. Each Python file is parsed **at most once**.
3. Elements are indexed by type once — after the first query, `classes()`, `functions()` & friends return immediately.
4. Counting code lines needs a second pass (`tokenize`), so it runs **only when a result shows line counts**.

> **Summary**
> Cheap work (validation) is eager. Expensive work (scan, parse, line count) is on demand, once. Chapter 13 shows how to refresh the cache after edits.

5 inspect() and location

One place — *all configuration happens in the* `inspect()` *function. Everything except* `location` *is optional and keyword-only.*

```
inspect(
    location,          # path of the project directory (required)
    *,
    depth="all",       # "all" or a non-negative integer
    extensions=None,   # None (= [".py"]), a list, or "all"
    exclude=None,      # None (= defaults), a list of globs, or []
    hidden=False,      # include dot-files and dot-directories?
) -> Project
```

Validated immediately

If something is wrong you get a precise, dedicated exception ***at the line that made the mistake****, not later when the first result is requested.*

location

A relative or absolute path to a ***directory****. Fou normalizes it internally. A path that does not exist, is not a directory, or cannot be accessed raises* `LocationError` *— you never see a raw* `FileNotFoundError`*.*

```
from fou import inspect

inspect("./my_project")        # relative
inspect("/home/you/code/app")  # absolute
inspect(".")                   # the current directory
```

Argument	Default	Meaning
`depth`	`"all"`	*how many directory levels to scan*
`extensions`	`None` *(= .py)*	*which files appear in the tree*
`exclude`	`None` *(= defaults)*	*globs that are never entered*
`hidden`	`False`	*include dot-files / dot-directories*

To focus on one file

`location` *must be a directory. Filter the result by* `element.path` *instead.*

5 depth

`depth` controls how many directory levels *below the root* are scanned. The default `"all"` scans everything.

depth	What is scanned
`0`	Only the files directly inside the root directory
`1`	The root plus one level of sub-directories
`2`	Two levels below the root, and so on
`"all"`	Everything (the default)

```
print(inspect("./shop", depth=0).files(show_lines=False))
```

```
Project: shop
├── __init__.py
├── config.py
└── main.py
```

```
print(inspect("./shop", depth=1).files(show_lines=False))
```

```
Project: shop
├── __init__.py
├── config.py
├── main.py
├── core/
│ ├── __init__.py
│ ├── models.py
│ └── payment.py
└── utils/
 └── helpers.py
```

Invalid depths

`-1`, `1.5`, `True` and the string `"2"` all raise **InvalidDepthError**. `True` is rejected on purpose, even though it is technically an integer in Python.

5 extensions and hidden

extensions

By default Fou looks at `.py` files only. Pass a list to include more, or `"all"` for everything. The leading dot is optional and matching is case-insensitive: `"py"`, `".py"` and `".PY"` are equivalent.

```
project = inspect("./shop", extensions=[".py", ".txt"], depth=0)
print(project.files())
```

```
Project: shop
├── __init__.py
│ -> 1 line
│
├── config.py
│ -> 4 lines
│
├── main.py
│ -> 7 lines
│
└── README.txt
 -> 2 non-blank lines
```

Discovery is not analysis

Extensions decide which files **appear in the tree**. Only `.py` files are analyzed for imports, functions, classes... A `.txt` or `.json` file shows up in `files()` and nowhere else. For non-Python text files the number means *non-blank lines*; binary files are marked `binary`.

hidden

Any file or directory whose name starts with a dot is skipped by default. Set `hidden=True` to include them. `exclude` still applies on top, so with `hidden=True` you will still not see `.git` unless you also clear the exclude list.

Summary

`extensions` = what appears · `hidden` = dot-names allowed · `exclude` = final filter on top of both.

5 exclude and determinism

`exclude` is a list of shell-style globs (`*`, `?`, `[seq]`) applied **during traversal**. An excluded directory is never entered, so excluding a huge folder like `node_modules` saves real time.

Pattern form	Meaning	Example
no `/`	matched against each file or directory name	`venv`, `*.egg-info`
contains `/`	matched against the path relative to the root	`docs/build`
trailing `/`	directories only	`build/`

The default list is `fou.DEFAULT_EXCLUDE`:

```
.git, .hg, .svn, __pycache__, .pytest_cache, .mypy_cache, .ruff_cache,
.tox, .nox, .eggs, *.egg-info, .venv, venv, node_modules, build, dist,
.idea, .vscode
```

- `exclude=None` (default) → uses `DEFAULT_EXCLUDE`.
- `exclude=[]` → excludes **nothing**.
- Any list you pass **replaces** the defaults. Extend instead: `[*fou.DEFAULT_EXCLUDE, "docs"]`.

```
project = inspect("./shop", exclude=[*fou.DEFAULT_EXCLUDE, "utils"])
```

```
Project: shop
├── __init__.py ├── config.py ├── main.py
└── core/ (__init__.py, models.py, payment.py)    # utils/ is gone
```

Replacing the defaults is a common surprise

`exclude=["docs"]` means you stopped excluding `.git`, `venv`, `__pycache__` — a venv inside your project will now be scanned. Extend the tuple instead.

Determinism rules

- Files come first, then directories; each group sorted case-insensitively.
- Symbolic links are **never followed** (no loops, no surprises).
- Directories with nothing to show are omitted.

6 Nine Types at a Glance

Overview — *Fou understands nine inspection types. Ask for one with its own method, or several at once with* `query()`*. The canonical list is* `fou.SUPPORTED_TYPES`*.*

```
import fou
print(fou.SUPPORTED_TYPES)
```

```
('files', 'imports', 'functions', 'async_functions', 'classes', 'methods',
 'variables', 'constants', 'decorators')
```

Type	*Method*	*What it lists*
files	`project.files()`	*the project tree, with code-line counts*
imports	`project.imports()`	*one entry per imported name*
functions	`project.functions()`	`def` *outside class bodies*
async_functions	`.async_functions()`	`async def` *outside class bodies*
classes	`project.classes()`	*class statements, nested included*
methods	`project.methods()`	`def` / `async def` *directly in a class*
variables	`project.variables()`	*bindings, with a scope*
constants	`project.constants()`	*UPPER_CASE or* `Final` *names*
decorators	`project.decorators()`	*each decorator, linked to its target*

1 · files

The file tree. Next to each file is its count of code lines (chapter 7). Pass `show_lines=False` *to skip counting entirely — that mode never reads a single file, so it is the fastest possible query.*

```
$ fou inspect ./shop --no-lines
```

6 imports

Every imported name becomes its own entry. *`from .core.models import Cart, Product` yields two entries — one for `Cart`, one for `Product` — which makes "who uses Cart?" easy to answer. Relative imports and `*` imports are handled too.*

```
$ fou inspect ./shop -t imports
Project: shop
Imports (10)
config.py
└── from typing import Final (line 2)
main.py
├── from .core.models import Cart (line 2)
├── from .core.models import Product (line 2)
└── from .utils.helpers import log_calls (line 3)
core/models.py
├── from dataclasses import dataclass (line 2)
├── from typing import List (line 3)
└── from ..config import TAX_RATE (line 5)
core/payment.py
├── import asyncio (line 2)
└── from .models import Cart (line 4)
utils/helpers.py
└── import functools (line 2)
```

With details: internal or external?

An import is `internal` when it is relative, or when its top-level module is one of the project's own top-level modules. Details also say whether it is conditional or lives inside an `if TYPE_CHECKING:` block.

```
$ fou inspect ./shop -t imports -d
config.py
└── from typing import Final (line 2)
 -> external
main.py
├── from .core.models import Cart (line 2)
| -> internal
...
```

Why it's useful

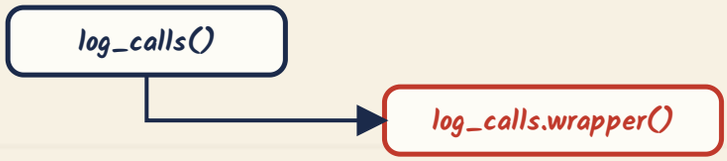
Recipe 14.1 uses the `internal` flag to list the external packages a project depends on.

6 functions and async_functions

`functions` lists every `def` that is **not directly in a class body**, nested ones included. Nested functions get a dotted qualified name (`log_calls.wrapper`) so you can tell them apart from top-level functions with the same simple name.

`async_functions` is the same for `async def`, and the output prefixes the name with `async`.

```
$ fou inspect ./shop -t functions,async_functions
Project: shop
Functions (5)
main.py
└── run() (line 7)
core/payment.py
└── refund() (line 12)
utils/helpers.py
├── slugify() (line 5)
├── log_calls() (line 9)
└── log_calls.wrapper() (line 11)
Async functions (1)
core/payment.py
└── async charge() (line 7)
```



the nested function's parent is the enclosing function

Where did my function go?

A `def` directly inside a class is a **method**, a class is its own type, and async functions live under `async_functions` — not under `functions`.

6 classes and methods

`classes` lists `class` statements, nested classes included. `methods` lists any function defined **directly inside a class body**. Use `is_async` to separate async methods, and `method_type` to see whether a method is an instance, class, static or property method.

```
$ fou inspect ./shop -t methods -d
Project: shop
Methods (5)
core/models.py
├── Product.with_tax(self) -> float (line 15)
│ -> 2 lines
│ -> doc: Return the price including tax.
├── Cart.__init__(self) -> None (line 23)
│ -> 2 lines
├── Cart.add(self, product: Product) -> None (line 26)
│ -> 2 lines
├── Cart.total(self) -> float (line 30)
│ -> 3 lines
│ -> decorators: @property
│ -> property method
└── Cart.empty_message() -> str (line 34)
 -> 3 lines
 -> decorators: @staticmethod
 -> static method
```

method_type	Looks like
instance	plain `def f(self)`
class	`@classmethod`
static	`@staticmethod`
property	`@property`

Summary

classes() tells you what exists; methods(details=True) tells you the signature, the line count and what kind of method each one is.

6 variables

Variables are classified by `scope`*, which makes this type more informative than a plain list of assignments:*

scope	*Meaning*	*Example in* **shop**
module	*bound at module level*	`__version__`
class	*bound in a class body*	`Product.name`
instance	`self.x = ...` *inside a method*	`Cart.items`
local	*bound inside a function*	`run.cart`

```
$ fou inspect ./shop -t variables
Project: shop
Variables (5)
__init__.py
└── __version__ (line 2)
 -> scope: module
main.py
└── run.cart (line 8)
 -> scope: local
core/models.py
├── Product.name (line 12)
│ -> scope: class
├── Product.price (line 13)
│ -> scope: class
└── Cart.items (line 24)
 -> scope: instance
```

Rules to remember

- *Repeated assignments to the same name in the same scope are* ***merged*** *into one entry (*`assignment_count` *says how many).*
- *A* `global x` *statement inside a function binds the module variable.*
- *Parameters, comprehension variables,* `except ... as` *names,* `match` *captures and lambda internals are intentionally* ***not*** *recorded.*

6 constants

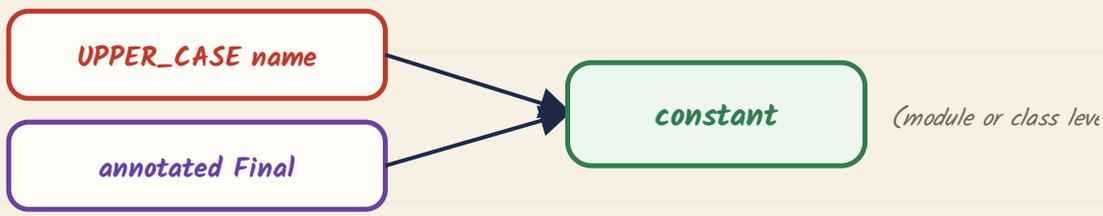
Python has no real constants, so Fou uses the convention everyone follows: a module- or class-level name matching `^_*[A-Z][A-Z0-9_]*$`, or any name annotated `Final`. Factories such as `TypeVar` are excluded. A constant never also appears under `variables`.

```
$ fou inspect ./shop -t constants
Project: shop
Constants (3)
config.py
├── DEBUG (line 4)
│ -> scope: module
├── MAX_ITEMS (line 5)
│ -> scope: module
└── TAX_RATE (line 6)
 -> scope: module
```

Notice MAX_ITEMS

`MAX_ITEMS: Final = 100` *is a constant because of its* ***Final*** *annotation.* `DEBUG` *and* `TAX_RATE` *are constants because of their* ***UPPER_CASE*** *names. Details mode shows* `final: true` *for the first one.*

Two routes to being a constant



6 decorators and combining types

*Each decorator is its **own entry**, linked to the class, function or method it decorates. The expression is shown exactly as written in the source, including call arguments.*

```
$ fou inspect ./shop -t decorators
Project: shop
Decorators (5)
main.py
└── @log_calls (line 6)
 -> on: function run
core/models.py
├── @dataclass (line 8)
| -> on: class Product
├── @property (line 29)
| -> on: method Cart.total
└── @staticmethod (line 33)
 -> on: method Cart.empty_message
utils/helpers.py
└── @functools.wraps(func) (line 10)
 -> on: function log_calls.wrapper
```

Combining types

*Pass several types — comma-separated on the command line, or a list in Python — to get **one result with one section per type**. Use `"all"` for all nine.*

```
$ fou inspect ./shop -t classes,constants
Project: shop
Classes (2)
core/models.py
├── Product (line 9)
└── Cart (line 20)
Constants (3)
config.py ... DEBUG, MAX_ITEMS, TAX_RATE (scope: module)
```

7 What details=True adds

Rule *— every inspection method accepts* `details=True`*. Details* ***never change which elements are returned****; they only enrich each element with extra information. What you get depends on the kind:*

kind	*Extra information with details*
function, method	*signature, parameters, returns, docstring, is_abstract, is_overload, method_type (methods), start_lineno, total_lines, code_lines, decorators, children*
class	*bases, keywords, docstring, start_lineno, total_lines, code_lines, decorators, children*
import	*statement, imported, alias, level, internal, bound_name, is_from, is_star, conditional, type_checking*
variable, constant	*value, annotation, annotation_only, assignment_count, defined_in (instance variables), final (constants)*
decorator	*expression, is_call, target_kind, target_qualname, arguments*

attrs versus details

attrs *= facts Fou always knows (e.g.* `module`*,* `level` *and* `internal` *for an import).* ***details*** *= the richer, optional layer you opt into. The JSON output merges both.*

Which calls pay for line counting?

Only definitions (functions, methods, classes, async functions) need the extra tokenize pass. `classes()`*,* `imports()` *and* `variables(details=True)` *skip it entirely.*

Summary

details=True = more fields per element, never more or fewer elements.

7 One method as structured data

This is exactly what `to_dict(details=True)` returns for `Cart.total` — and what ends up in the JSON file in chapter 10. (Two small blocks are collapsed to one line to save space.)

```
method = list(inspect("./shop").methods(details=True))[3]
print(json.dumps(method.to_dict(details=True), indent=2))
```


```
{
  "id": "core/models.py::Cart.total",
  "kind": "method",
  "name": "total",
  "qualname": "Cart.total",
  "fqn": "shop.core.models.Cart.total",
  "path": "core/models.py",
  "module": "shop.core.models",
  "location": {"lineno": 30, "col": 4, "end_lineno": 31, "end_col": 52},
  "parent": "core/models.py::Cart",
  "is_async": false,
  "signature": "(self) -> float",
  "parameters": [{"name": "self", "kind": "positional_or_keyword",
                  "annotation": null, "default": null}],
  "returns": "float",
  "docstring": null,
  "is_abstract": false,
  "is_overload": false,
  "method_type": "property",
  "start_lineno": 29,
  "total_lines": 3,
  "code_lines": 3,
  "decorators": ["property"],
  "children": []
}
```


Spot the numbers

`start_lineno` *is* **29** *(the* `@property` *line) while* `location.lineno` *is* **30** *(the* `def` *line). A decorated definition's span starts at its first decorator.*

7 Parameters and Code Lines

Parameters in detail

The `parameters` list describes every parameter: its `kind` (`positional_or_keyword`, `keyword_only`, `var_positional`, ...), its annotation and its default value as source text.

```
for fn in inspect("./shop").functions(details=True):
    if fn.name == "refund":
        print("signature :", fn.details["signature"])
        for p in fn.details["parameters"]:
            print("parameter :", p)
        print("returns   :", fn.details["returns"])
```

```
signature : (amount: float) -> str
parameter : {'name': 'amount', 'kind': 'positional_or_keyword',
             'annotation': 'float', 'default': None}
returns   : str
```

What counts as a line of code?

When Fou prints `-> 94 lines` it means 94 lines of actual code. The number comes from Python's `tokenize` module, not from counting newlines — much more honest than `wc -l`.

Line type	Counted?
Blank lines	no
Comment-only lines	no
Docstring lines	no
Every physical line of a multi-line expression, and of a non-docstring string literal	**yes**
A decorated definition's span	starts at its first decorator

Why not just wc -l?

Your editor counts physical lines. Fou's code-line count is a better measure of how big a file or function really is.

7 LineStats and Lazy Counting

The full breakdown for any Python file is a `LineStats` *object with five numbers:* `total`, `code`, `blank`, `comment`, `docstring`.

```
for f in inspect("./shop").files().files:
    if f.lines and f.lines.total:
        s = f.lines
        print(f"{f.path:<20} total={s.total:<3} code={s.code:<3} "
              f"blank={s.blank:<3} comment={s.comment:<3} docstring={s.docstring}")
```

```
__init__.py          total=2   code=1   blank=0   comment=0   docstring=1
config.py            total=6   code=4   blank=1   comment=0   docstring=1
main.py              total=10  code=7   blank=2   comment=0   docstring=1
core/models.py       total=35  code=20  blank=11  comment=0   docstring=4
core/payment.py      total=13  code=7   blank=5   comment=0   docstring=1
utils/helpers.py     total=13  code=8   blank=4   comment=0   docstring=1
```

Look at `core/models.py`*: far more physical lines than the* ***20 lines of code****. Docstrings and blank lines account for the difference — which is why a code-line count is a better size measure.*

Non-Python files

Fou knows nothing about comments in other languages, so it reports ***non-blank lines*** *instead, labeled as such, and* `LineStats.precise` *is* `False`*. Binary files are detected and shown as* `binary`*.*

Line counting is lazy

A tokenize pass costs about as much as parsing, so Fou only runs it when you actually look at line counts: in `files()` *(unless* `show_lines=False`*) and with* `details=True` *on functions, methods, classes and async functions.* ***A file is never counted twice.***

8 The InspectionResult

Daily driver *— every inspection method returns an* ***InspectionResult****. It behaves like a read-only collection:*

You write	*You get*
`len(result)`	*total number of items across all requested types*
`for item in result`	*each item, grouped by type in the order requested*
`result["classes"]`	*the list of items for one type (*`KeyError` *if not requested)*
`result.types`	*tuple of requested type names*
`result.files`	*only the FileModel items (for* `files()` *results)*
`result.elements`	*only the Element items*
`result.errors`	*problems met along the way (the scan continues past them)*
`str(result)`	*human-readable text, no links*
`result.render(...)`	*the same text as a string, optionally with clickable links*
`result.show(...)`	*prints the text; links added automatically on a terminal*
`.to_dict()` / `.to_json()`	*structured data / a JSON string*
`result.save(format="json", path=...)`	*writes the JSON to a file*

```
result = inspect("./shop").query(type=["classes", "constants"])
print(repr(result))
print(result.types)
print(len(result["classes"]), "classes,", len(result["constants"]), "constants")
print([c.name for c in result["constants"]])
```

```
<InspectionResult types=['classes', 'constants'] items=5 errors=0>
('classes', 'constants')
2 classes, 3 constants
['DEBUG', 'MAX_ITEMS', 'TAX_RATE']
```

8 Anatomy of an Element

*Every discovered code item is an **Element**. These are the attributes you will use most:*

Attribute	Meaning
`kind`	*import, function, method, class, variable, constant or decorator*
`name`	*the simple name (`total`)*
`qualname`	*dotted name inside the file (`Cart.total`)*
`module`	*dotted module of the file (`shop.core.models`)*
`path`	*file path relative to the project root, always with `/`*
`location`	*lineno, col, end_lineno, end_col*
`parent`	*the enclosing element, or `None` at top level*
`children`	*elements defined directly inside this one*
`decorators`	*decorator elements attached to this one*
`is_async`	`True` *for* `async def`
`scope`	*for variables, constants and imports*
`attrs`	*always-present extra facts (e.g. `internal` on imports)*
`details`	*extra facts only when you asked for* `details=True`
`id`	*stable identifier such as* `core/models.py::Cart.total`

A helpful KeyError

Asking for a type you did not request gives a message, not a mystery:

```
result = inspect("./shop").classes()
try:
    result["functions"]
except KeyError as error:
    print(error)
```

```
"'functions' was not requested; this result holds: classes"
```

8 Walking the Tree and Printing

Walking the tree

Elements know their neighbours. `children` goes down; `parent` and `ancestors()` go up; `walk()` visits an element and everything below it (parents before children).

```
wrapper = [f for f in project.functions() if f.qualname == "log_calls.wrapper"][0]
print("element  :", wrapper)
print("parent   :", wrapper.parent)
print("ancestors:", [a.qualname for a in wrapper.ancestors()])
cart = project.classes()["classes"][1]
print("walk     :", [e.qualname for e in cart.walk()])
```

```
element  : <Element function 'log_calls.wrapper' utils/helpers.py:11>
parent   : <Element function 'log_calls' utils/helpers.py:9>
ancestors: ['log_calls']
walk     : ['Cart', 'Cart.__init__', 'Cart.items', 'Cart.add', 'Cart.total',
            'Cart.empty_message']
```

Printing, rendering, showing

Method	Returns / does	Links
`str(result)`	*returns text*	*never*
`result.render(links=False)`	*returns text*	*only if you pass `links=True` or `"auto"`*
`result.show(links="auto")`	*prints text*	*automatically, on an interactive terminal*

The split is deliberate: pipes, files and tests stay clean because `str()` never contains escape sequences, while interactive use gets clickable output for free.

Filtering and sorting

A result is just an iterable, so ordinary Python does the rest — e.g. all property methods, sorted by name:

```
methods = inspect("./shop").methods(details=True)
props = [m for m in methods if m.details["method_type"] == "property"]
for m in sorted(props, key=lambda m: m.qualname):
    print(f"{m.module}.{m.qualname}", "->", m.details["returns"])
```

```
shop.core.models.Cart.total -> float
```

9 The Idea and OSC 8

Idea — *reading a list of classes is useful; clicking one and landing on its line in your editor is better. Fou can turn every file name and every element in its terminal output into a hyperlink that opens the file at that exact line.*

Here is Fou inspecting its own source. In a real terminal each file name and each `(line N)` *is a link; clicking* `read_plain_text()` *opens* `reader.py` *on line 44.*

```
inspection/reader.py
├── _normalize_newlines() (line 19)
├── _read_bytes() (line 23)
├── read_python_source() (line 31)
└── read_plain_text() (line 44)
```

How it works: OSC 8 hyperlinks

Modern terminals understand an escape sequence called ***OSC 8*** *that turns any text into a link:*

```
ESC ] 8 ; ; URL ESC \ visible text ESC ] 8 ; ; ESC \
```

Fou wraps every file name and element in such a link. Which program opens it, and whether it jumps to a line, is decided ***entirely by the URL****, built from a small template called a* *scheme*. *Ask for a linked string and print its* `repr()` *to see the invisible characters:*

```
text = inspect("./shop").classes().render(links=True, scheme="vscode")
for line in text.splitlines()[3:6]:
    print(repr(line))
```

```
''
'\x1b]8;;vscode://file/home/you/projects/shop/core/models.py:1:1\x1b\\core/models.py\x1b
'├── \x1b]8;;vscode://file/.../core/models.py:9:1\x1b\\Product (line 9)\x1b]8;;\x1b\\'
```

Reading it

`vscode://file/.../core/models.py:9:1` *is the* ***link target****;* `Product (line 9)` *is the* ***visible text****. The terminal hides the escape characters and underlines the text on hover.*

9 Schemes and Link Modes

Schemes: choosing where a click goes

scheme	Opens	Jumps to line?
`file` (default)	the OS default program for the file	**no** (file:// has no line)
`vscode`	Visual Studio Code	yes
`vscode-insiders`	VS Code Insiders	yes
`cursor`	Cursor	yes
`pycharm`	PyCharm	yes
`idea`	IntelliJ IDEA	yes
`sublime`	Sublime Text	yes
`textmate`	TextMate	yes

> **The default scheme does not jump to a line**
> If you do nothing the scheme is `file`: it opens the file but cannot say which line. Pick an editor scheme (e.g. `vscode`) for line-accurate jumps.

The three link modes

links	Behaviour
`"auto"` (default for `show()` and the CLI)	links only when the output is an interactive terminal and TERM is not `dumb`
`"always"` or `True`	always emit links
`"never"` or `False`	never emit links

In auto mode Fou checks `stream.isatty()` — so the same command prints clean text into a file or pipe, and clickable text when run by hand.

9 Custom Templates and Typos

Custom templates

Editor not on the list? Pass a template string. Fou fills in these placeholders:

Placeholder	Replaced by
`{path}`	absolute path, percent-encoded, always starting with `/`
`{fspath}`	the same path without forcing a leading `/`
`{uri}`	a `file://` URI
`{line}`	line number, 1-based
`{col}`	column number, 1-based

```
result.show(scheme="vscode://file{path}:{line}:{col}")
result.show(scheme="myeditor://open?file={fspath}&line={line}")
```

Typos fail loudly (with a hint)

```
try:
    result.render(links=True, scheme="vscod")
except FouError as error:
    print(type(error).__name__); print(error)
```

```
InvalidOptionError
Unknown link scheme 'vscod'. Did you mean 'vscode'? Built-in schemes: file,
vscode, vscode-insiders, cursor, pycharm, idea, sublime, textmate; or pass a
template such as 'vscode://file{path}:{line}:{col}'.
```

From Python

```
result = project.classes()
result.show()                                  # links on a terminal, plain in a pipe
result.show(scheme="vscode")                   # click -> VS Code at the line
result.show(links="always", scheme="pycharm")  # force links on
text = result.render(links=True, scheme="vscode")  # a string with the links inside
```

9 Links from the Command Line

Consistency — *Fou validates the scheme even when links end up disabled (e.g. output to a pipe). A typo fails the same way everywhere, instead of hiding until someone runs it in a real terminal.*

From the command line

```
$ fou inspect . -t methods --editor vscode
$ fou inspect . --links always --editor "vscode://file{path}:{line}:{col}"
```

Rather than typing `--editor` *every time, set a default in your shell profile:*

Environment variable	Effect
`FOU_LINK_SCHEME`	*default scheme, e.g.* `vscode` *(used when none is given)*
`FOU_LINKS`	`always` *or* `never` *— overrides* `links="auto"`

```
$ export FOU_LINK_SCHEME=vscode
```

Keep CI output clean

```
$ FOU_LINKS=never fou inspect . -t classes > classes.txt
$ fou inspect . --links never --json | python -m json.tool
```

> **Remember**
>
> *In* `auto` *mode a pipe or file already gets plain text. Use* `never` *to be explicit in scripts,* `always` *to force links in a terminal that Fou cannot detect.*

9 Terminals, Limits and Internals

Which terminals support OSC 8?

Widely supported: **iTerm2**, **Windows Terminal**, **the VS Code integrated terminal**, **GNOME Terminal** (and other VTE-based), **kitty**, **WezTerm**, **Alacritty**, **foot**.

Two caveats

- macOS **Terminal.app** does not support OSC 8.
- Inside **tmux**, links need tmux 3.4 or newer.

How to click

Most terminals need a modifier key: usually **Ctrl+click** on Linux/Windows or **Cmd+click** on macOS. Hovering normally shows the link target.

What links never touch

- `str(result)` and plain `result.render()` contain no links unless you ask.
- The JSON output **never** contains links.
- Stripping the escape sequences from a linked rendering gives *exactly* the plain rendering — tests and diffs stay stable.
- Directories are not linked (there is no line to jump to).

Under the hood

The logic lives in `fou.inspection.links`: **LinkBuilder** turns a relative path + line into a URL, and **hyperlink()** wraps text in the OSC 8 sequence. Internal, but simple enough to use directly:

```
from fou.inspection.links import LinkBuilder, hyperlink

builder = LinkBuilder("/home/you/projects/shop", scheme="vscode")
url = builder.url("core/models.py", lineno=9, col=0)
print(url)
print(repr(hyperlink("Product (line 9)", url)))
```

```
vscode://file/home/you/projects/shop/core/models.py:9:1
'\x1b]8;;vscode://file/home/you/projects/shop/core/models.py:9:1\x1b\\Product (line 9)\x
```

10 Three Ways to Get Data Out

Goal — hand Fou's results to scripts, CI jobs and other tools.

Call	Returns
`result.to_dict()`	a plain Python dictionary (JSON-compatible)
`result.to_json()`	the same data as a JSON string
`result.save(format="json", path=...)`	writes a JSON file and returns its path

```
result = inspect("./shop").query(type=["classes", "methods"], details=True)
saved = result.save(format="json", path="./structure.json")
print("saved to:", saved)
```

```
saved to: /home/you/projects/structure.json
```

save() has three guarantees

1. `format` is validated — anything other than `"json"` raises **InvalidSaveFormatError**.
2. The parent directory of `path` must already exist, otherwise **InvalidSavePathError**.
3. The write is **atomic**: you never end up with a half-written file if something goes wrong.

JSON is for machines, text is for humans

`save()` writes structured data, never terminal text. The JSON is the same whether or not you used links, `show_lines` or a particular editor scheme.

Safe to commit

Because output is deterministic, a saved result can go in version control and two results can be compared with a plain `diff`.

10 The Schema

The document has a stable layout, versioned by `schema_version` *(currently* `"1.0"`*):*

```
{ schema_version, tool, tool_version, module,
  project  { name, root, options },
  query    { types, details, show_lines },
  summary  { files, directories, counts, errors },
  tree     { ...nested directories and files... },  # only when "files" requested
  elements { <type>: [ { id, kind, name, qualname, fqn, path, module,
                         location { lineno, col, end_lineno, end_col },
                         parent, scope, ... } ] },
  errors   [ { path, phase, type, message, lineno, col } ] }
```

Look at a real document

```
data = inspect("./shop").query(type=["classes", "functions"]).to_dict()
print(list(data))
print(json.dumps(data["summary"], indent=2))
print(json.dumps(data["query"]))
```

```
['schema_version', 'tool', 'tool_version', 'module', 'project', 'query',
 'summary', 'elements', 'errors']
{
  "files": 7,
  "directories": 2,
  "counts": {"classes": 2, "functions": 5},
  "errors": 0
}
{"types": ["classes", "functions"], "details": false, "show_lines": true}
```

Reading the schema

summary *gives quick counts;* ***elements*** *groups items by type;* ***errors*** *is the recorded-not-raised list from chapter 12.*

10 Stable Ids and Reading Back

Stable ids and relationships

Relationships are expressed through *id strings*, not nesting. An id is `path::qualname`, e.g. `core/models.py::Cart.total`. When the same qualname appears more than once in a file (a function redefined later), repeats get `#2`, `#3`, ...

- `parent` = id of the enclosing element, or `null`.
- With `details=True`, `children` lists direct descendants as `{id, kind, name}`; decorators attach to what they decorate.
- Paths are relative with `/` separators on every OS; `col` is 0-based as in Python's `ast`.

```
cls = inspect("./shop").classes(details=True).to_dict()["elements"]["classes"][1]
keys = ("id", "fqn", "parent", "decorators", "children")
print(json.dumps({k: cls[k] for k in keys}, indent=2))
```

```
{
  "id": "core/models.py::Cart",
  "fqn": "shop.core.models.Cart",
  "parent": null,
  "decorators": [],
  "children": [
    {"id": "core/models.py::Cart.__init__", "kind": "method", "name": "__init__"},
    {"id": "core/models.py::Cart.items", "kind": "variable", "name": "items"},
    {"id": "core/models.py::Cart.add", "kind": "method", "name": "add"},
    ...
  ]
}
```

Reading the file back

It is ordinary JSON, so any language can consume it. In Python:

```
with open("structure.json") as handle:
    data = json.load(handle)
for cls in data["elements"]["classes"]:
    where = cls["location"]["lineno"]
    print(cls["fqn"], "line", where, "-", cls["code_lines"], "code lines")
```

```
shop.core.models.Product line 9 - 6 code lines
shop.core.models.Cart line 20 - 11 code lines
```

11 Synopsis and Options

```
fou [--version] inspect [location] [options]
python -m fou ...        # identical
```

*At the moment there is **one subcommand**, `inspect`. `location` defaults to `.` (the current directory).*

Option	Meaning	Python equivalent
`--depth N`	`all` (default) or a non-negative integer	`depth=`
`-e, --ext EXT`	extension to include; repeatable; or `all`; default `.py`	`extensions=`
`-x, --exclude PATTERN`	glob to exclude; repeatable; **replaces** the default list	`exclude=`
`--hidden`	include hidden files and directories	`hidden=True`
`-t, --type TYPE`	type(s); repeatable or comma-separated; or `all`; default `files`	`query(type=)`
`-d, --details`	show additional detail	`details=True`
`--no-lines`	hide code-line counts in the tree	`show_lines=False`
`--json`	print JSON instead of text	`result.to_json()`
`--links MODE`	`auto` (default), `always`, `never`	`links=`
`--editor SCHEME`	link scheme or template	`scheme=`
`-o, --output PATH`	also save the result as JSON	`result.save()`

Exit codes

Code	Meaning
`0`	success
`2`	a Fou error (bad location, option, type...) or a command-line syntax error

11 Help and First Examples

```
$ fou inspect --help
usage: fou inspect [-h] [--depth DEPTH] [-e EXT] [-x PATTERN] [--hidden] [-t TYPE]
                   [-d] [--no-lines] [--json] [--links {auto,always,never}]
                   [--editor SCHEME] [-o PATH]
                   [location]

positional arguments:
  location              Project directory (default: .)

options:
  --depth DEPTH         "all" (default) or a non-negative integer.
  -e EXT, --ext EXT     Extension to inspect, repeatable (e.g. -e .py -e .json)
                        or "all". Default: .py
  -x PATTERN, --exclude PATTERN
                        Glob to exclude, repeatable. Replaces the default
                        exclude list.
  --hidden              Include hidden files and directories.
  -t TYPE, --type TYPE  Inspection type(s), repeatable or comma-separated, or
                        "all". Default: files.
  -d, --details         Show additional detail.
  --no-lines            Hide line counts in the file tree.
  --json                Print JSON instead of text.
  --links {auto,always,never}
                        Make files and elements clickable terminal hyperlinks.
                        'auto' (default) only on an interactive terminal; env
                        FOU_LINKS=always|never overrides auto.
  --editor SCHEME       Where a click goes: file (default), vscode, cursor,
                        pycharm, idea, sublime, textmate, or a template like
                        'vscode://file{path}:{line}:{col}'. Default also from
                        env FOU_LINK_SCHEME.
  -o PATH, --output PATH
                        Also save the result as JSON.
```

Top level only

```
$ fou inspect ./shop --depth 0 --no-lines
Project: shop
├── __init__.py
├── config.py
└── main.py
```

Text files in the tree, one folder excluded

```
$ fou inspect ./shop -e .py -e .txt -x utils --depth 0
...
└── README.txt
 -> 2 non-blank lines
```

11 More CLI Examples

Several types, with details

`-t` may be repeated or comma-separated:

```
$ fou inspect ./shop -t async_functions -t decorators -d
Project: shop
Async functions (1)
core/payment.py
└── async charge(cart: Cart) -> bool (line 7)
 -> 3 lines
Decorators (5)
main.py
└── @log_calls (line 6)
 -> on: function run
...
```

Machine-readable output

`--json` replaces the text; `-o` additionally writes the file:

```
$ fou inspect ./shop -t constants --json -o constants.json
{
  "schema_version": "1.0",
  "tool": "fou",
  "tool_version": "0.1.0",
  "module": "inspect",
  ...
```

stderr vs stdout

The line `Saved: ...` goes to the **standard error** stream, so it never contaminates JSON piped from standard output.

Errors look the same everywhere

```
$ fou inspect ./shop -t clases
fou: error: Unsupported inspection type 'clases'. Did you mean 'classes'?
```

12 The Exception Hierarchy

Two situations — *Fou fails* ***loudly for mistakes*** *and* ***quietly for messy projects****. A bad argument raises a clear exception immediately. A messy project (unreadable folder, syntax error) is recorded, and the scan keeps going — one broken file never hides the other four hundred.*

Every error Fou raises on purpose derives from `FouError`*, so one* `except FouError` *catches them all. Option errors also derive from* `ValueError`*, which feels natural in Python code.*

```
FouError
├── LocationError            location missing / not a directory / inaccessible
├── InvalidOptionError       (also ValueError)
│   ├── InvalidDepthError           depth is not "all" or a non-negative int
│   ├── InvalidExtensionsError      extensions is not "all" or a non-empty list
│   ├── InvalidExcludeError         exclude holds an unusable pattern
│   ├── InvalidInspectionTypeError  a requested type is malformed/unsupported
│   ├── InvalidSaveFormatError      (also SaveError)
│   └── InvalidSavePathError        (also SaveError)
└── SaveError                a result could not be saved
```

The hierarchy as defined in `fou/core/errors.py`*.*

Argument errors vs project problems

Situation	***Behaviour***
Bad location	`LocationError`
Bad depth / extensions / exclude / type	`InvalidDepthError`, `InvalidExtensionsError`, `InvalidExcludeError`, `InvalidInspectionTypeError`
Bad save format or path	`InvalidSaveFormatError` *or* `InvalidSavePathError`
Unreadable directory, undecodable or syntactically invalid file	***not raised*** *— recorded in* `result.errors`

Errors as data

Mistakes in your arguments produce exceptions. Problems in the project being inspected produce ErrorInfo entries. The two never mix.

12 Errors in Practice

Raised immediately

```
attempts = [
    dict(location="./nowhere"),
    dict(location="./shop", depth=-1),
    dict(location="./shop", depth=1.5),
]
for kwargs in attempts:
    try:
        inspect(**kwargs)
    except FouError as error:
        print(f"{type(error).__name__}: {error}")
```

```
LocationError: Project location does not exist: /home/you/projects/nowhere
InvalidDepthError: Invalid depth -1: depth cannot be negative.
InvalidDepthError: Invalid depth 1.5: use "all" or a non-negative integer.
```

Recorded, not raised

One good file and one with a syntax error, then ask for classes:

```
Path("broken_demo/good.py").write_text("class Good:\n pass\n")
Path("broken_demo/bad.py").write_text("def oops(:\n pass\n")

result = inspect("./broken_demo").classes()
print(len(result), "class found:", [c.name for c in result])
for problem in result.errors:
    print(problem)
    print(problem.phase, problem.type, problem.lineno, problem.col)
```

```
1 class found: ['Good']
bad.py: SyntaxError: invalid syntax (line 1)
parse SyntaxError 1 10
```

- *ErrorInfo has five fields:* `path`, `phase` *(scan / read / parse)*, `type`, `message`, `lineno` / `col` *where available. Errors also appear in the JSON under* `errors`.
- *A syntax-error file yields* ***no elements****; undecodable text is phase* `read`*; an unreadable directory is phase* `scan`.

Treat result.errors as a health check

In automation, `assert not result.errors` *is a one-line way to make a pipeline fail when the project contains Python that does not even parse.*

13 Scan Once, Parse Once

Why it's fast — *Fou does cheap work eagerly and expensive work once, on demand.*

- *The filesystem is scanned* ***once****, on the first inspection call.*
- *Each file is read and parsed* ***at most once****; all methods share that cache.*
- *Elements are indexed by type once per scan, so later* `classes()` / `imports()` *calls return immediately.*

Lazy line counting

Call	Counts lines?
`files()`	*yes (unless* `show_lines=False`*)*
`files(show_lines=False)`	***no*** *— reads no file contents at all*
`functions(details=True)`, `methods(...)`, `classes(...)`, `async_functions(...)`	*yes*
`classes()`, `imports()`, `variables(details=True)`, …	*no*

A file is never counted twice, however often you ask.

Refreshing after edits

The cache is a ***snapshot****. If files change while your program runs, call* `project.refresh()`*: it rescans the filesystem and re-analyzes only files whose size or modification time changed.*

```
Path("scratch/a.py").write_text("def first():\n return 1\n")
project = inspect("./scratch")
print("before :", len(project.functions()))
Path("scratch/b.py").write_text("def second():\n return 2\n")
print("cached :", len(project.functions()))   # still 1: from the cache
project.refresh()
print("refresh:", len(project.functions()))   # now 2
```

```
before : 1
cached : 1
refresh: 2
```

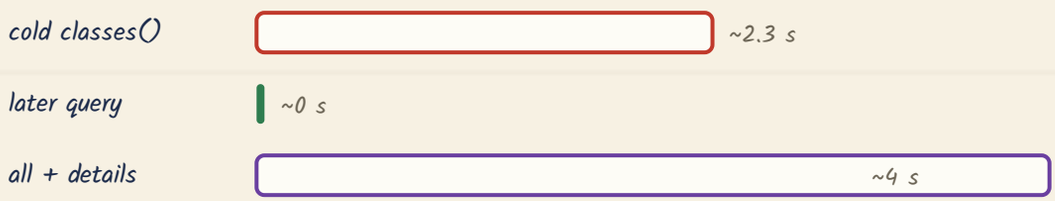
13 Numbers and Speed Tips

Measured numbers

The project's own benchmark ran on the Python standard library: **515 files, about 48,000 elements**, one core, Python 3.12, best of three.

Operation	Time
Cold `classes()` (includes about 1.0 s of `ast.parse`)	about 2.3 s
Any later query	about 0 s
Counting lines for everything, once	about +1.4 s
Full cold `query(type="all", details=True)`	about 4 s
`result.save()` of that 45 MB JSON	about +1.4 s

Treat these as orders of magnitude — your hardware and Python version will differ.



rough sketch of the table above (not to scale)

Six practical speed tips

1. **Reuse the Project.** Create it once; call several methods on it.
2. **Exclude what you don't need.** Excluded directories are never entered — the single biggest win.
3. **Limit depth** when you only need the top of a large tree.
4. Use `files(show_lines=False)` when you only need names — it never reads file contents.
5. Ask for `details=True` only when you use it (it adds line counting).
6. Write JSON only when you need it — serializing a huge result is pure-Python work.

14 Recipes 1–3: Imports, Docstrings, Long Functions

Pattern: *create a Project, ask for the types you need, then use plain Python. All run on the tutorial project.*

14.1 Which third-party packages does my project import?

```
from collections import Counter
external = Counter()
for imp in project.imports(details=True):
    if not imp.attrs.get("internal"):
        top = (imp.attrs.get("module") or imp.name).split(".")[0]
        external[top] += 1
for package, count in external.most_common():
    print(f"{package:<12} {count}")
```

```
typing       2
dataclasses  1
asyncio      1
functools    1
```

14.2 Classes and functions without a docstring

```
missing = [el for el in project.query(type=["classes", "functions", "methods"],
                                      details=True)
           if not el.details.get("docstring")]
for el in missing:
    print(f"{el.path}:{el.location.lineno} {el.kind:<8} {el.qualname}")
print(f"\n{len(missing)} definitions without a docstring")
```

```
main.py:7 function run
core/payment.py:12 function refund
...                                        (9 definitions in total)
```

14.3 The longest functions

```
defs = list(project.query(type=["functions", "methods"], details=True))
defs.sort(key=lambda el: el.details["code_lines"], reverse=True)
for el in defs[:3]:
    n = el.details["code_lines"]
    print(f"{n:>3} lines {el.path}:{el.location.lineno} {el.qualname}")
```

```
  5 lines main.py:7 run
  5 lines utils/helpers.py:9 log_calls
```

14 Recipes 4–5: CI Limit and CSV Export

14.4 Enforce a function-length limit in CI

Exit non-zero when a function is too long and your CI fails the build. The limit is deliberately tiny so the tutorial project trips it.

```
import sys
LIMIT = 4
too_long = [el for el in project.query(type=["functions", "methods"], details=True)
            if el.details["code_lines"] > LIMIT]
for el in too_long:
    print(f"{el.path}:{el.location.lineno} {el.qualname}: "
          f"{el.details['code_lines']} code lines (max {LIMIT})")
sys.exit(1 if too_long else 0)
```

```
main.py:7 run: 5 code lines (max 4)
utils/helpers.py:9 log_calls: 5 code lines (max 4)
```

Make it clickable

Output in the form `path:line` *is understood by most terminals and editors as a jump target — even without Fou's own links.*

14.5 Export every element to CSV

```
import csv
with open("elements.csv", "w", newline="") as handle:
    writer = csv.writer(handle)
    writer.writerow(["kind", "name", "file", "line"])
    for el in project.query(type="all"):
        if hasattr(el, "kind"):          # skip FileModel items
            writer.writerow([el.kind, el.qualname, el.path, el.location.lineno])
```

```
kind,name,file,line
import,Final,config.py,2
import,Cart,main.py,2
import,Product,main.py,2 ...
```

Why hasattr?

`query(type='all')` *includes FileModel items too (the file tree); only Elements have a* `kind` *and a* `location`.

14 Recipes 6–8: Summary, Overview, Async Inventory

14.6 A per-file summary

```
table = defaultdict(lambda: defaultdict(int))
for el in project.query(type=["classes", "functions", "async_functions", "methods"]):
    label = "async function" if (el.is_async and el.kind == "function") else el.kind
    table[el.path][label] += 1
for path in sorted(table):
    summary = ", ".join(f"{n} {kind}" for kind, n in sorted(table[path].items()))
    print(f"{path:<18} {summary}")
```

```
core/models.py     2 class, 5 method
core/payment.py    1 async function, 1 function
main.py            1 function
utils/helpers.py   3 function
```

14.7 Class overview with bases, decorators and methods

```
for cls in project.classes(details=True):
    methods = [c.name for c in cls.children if c.kind == "method"]
    decorators = [d.attrs["expression"] for d in cls.decorators]
    print(f"{cls.qualname} bases={cls.details['bases']} decorators={decorators}")
    print(f"  methods: {', '.join(methods)}")
```

```
Product bases=[] decorators=['dataclass']
  methods: with_tax
Cart bases=[] decorators=[]
  methods: __init__, add, total, empty_message
```

14.8 Inventory of async code

```
for fn in project.async_functions(details=True):
    print(f"{fn.module}.{fn.qualname} ({fn.path}:{fn.location.lineno})")
    print("  signature:", fn.details["signature"])
```

```
shop.core.payment.charge (core/payment.py:7)
  signature: (cart: Cart) -> bool
```

14 Recipes 9–10: API Index and Version Diff

14.9 Generate a Markdown API index

```
lines = ["# API index", ""]
for cls in project.classes(details=True):
    doc = cls.details["docstring"] or "No docstring"
    where = f"{cls.path}:{cls.location.lineno}"
    lines.append(f"- `{cls.module}.{cls.qualname}` ({where}): {doc}")
    for child in cls.children:
        if child.kind == "method":
            lines.append(f"  - `{child.name}`")
print("\n".join(lines))
```

```
# API index

- `shop.core.models.Product` (core/models.py:9): A product that can be sold.
  - `with_tax`
- `shop.core.models.Cart` (core/models.py:20): A shopping cart.
  - `__init__`   - `add`   - `total`   - `empty_message`
```

14.10 What changed between two versions?

Element ids are stable, so comparing two snapshots is a set difference. A function moves into a new class — Fou reports exactly what appeared and disappeared.

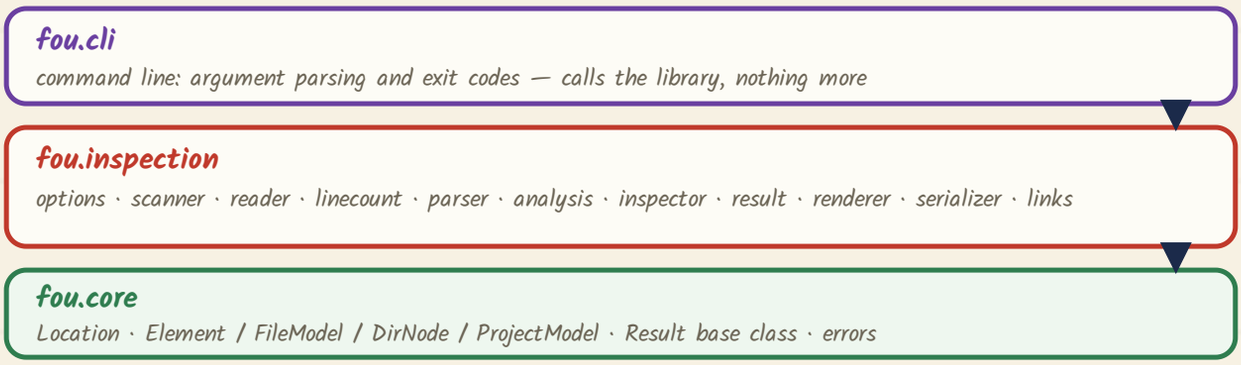
```
def ids(project):
    data = project.query(type=["classes", "functions", "methods"]).to_dict()
    return {e["id"] for group in data["elements"].values() for e in group}

# scratch/app.py:  def load / def save     -> snapshot "before"
before = ids(inspect("./scratch"))
# rewrite app.py: def load / class Store: def save(self)
project.refresh()
after = ids(project)
print("added  :", sorted(after - before))
print("removed:", sorted(before - after))
```

```
added  : ['app.py::Store', 'app.py::Store.save']
removed: ['app.py::save']
```

15 Layers and Modules

Three layers *— each depends only on the layer beneath it, so the shared model can serve many future modules.*



arrows point from a layer to the layer it depends on

What each module does

Module	Responsibility
`core/location.py`	*Location: line, column, end line, end column*
`core/models.py`	*Element, FileModel, DirNode, ProjectModel, LineStats, ErrorInfo*
`core/result.py`	*abstract Result base: render, show, to_json, atomic save*
`core/errors.py`	*the FouError hierarchy*
`inspection/options.py`	*validation and normalization of every user-facing argument*
`inspection/scanner.py`	*walks the directory, builds the file tree*
`inspection/reader.py`	*reads sources, normalizes newlines, detects binary*
`inspection/linecount.py`	*tokenize-based code / blank / comment / docstring classifier*
`inspection/parser.py`	***largest module**: ast nodes → Element objects*
`inspection/analysis.py`	*connects reading, parsing and counting for one file*
`inspection/inspector.py`	`inspect()` *and Project: caching, indexing, refresh()*
`inspection/result.py`	*InspectionResult*
`inspection/renderer.py`	*terminal text, with optional links*
`inspection/serializer.py`	*JSON structure (schema_version 1.0)*
`inspection/links.py`	*OSC 8 hyperlinks, schemes, templates*
`_version.py`	*the version — single source of truth for pyproject.toml*

15 Design Decisions and New Modules

Design decisions worth knowing

- ***Separation of concerns.*** *The model knows nothing about printing or JSON; a new output format = a new serializer.*
- ***Errors as data.*** *Messy projects produce ErrorInfo; mistakes in arguments produce exceptions. They never mix.*
- ***Determinism first.*** *Explicit sorting, symlinks never followed; output never depends on filesystem order.*
- ***Laziness with caching.*** *Cheap work (validation) eager; expensive work (scan, parse, count) on demand, once.*
- ***Single version source.*** `_version.py` *is read by pyproject through* `[tool.setuptools.dynamic]`*.*
- ***Typed package.*** *The empty* `py.typed` *marker tells type checkers Fou ships inline types.*

Adding a new module

Future modules (`find`*,* `relation`*) are meant to consume the existing project model and return a subclass of* `fou.core.Result`*. A subclass implements only* `to_dict()` *and* `render()`*;* `show()`*,* `to_json()` *and* `save()` *come for free — atomic write and format validation included. (An illustrative sketch, not part of the current release.)*

```
from typing import Any, Dict
from fou.core import Result

class CountResult(Result):
    """A tiny example result: how many elements of each kind were found."""
    def __init__(self, counts: Dict[str, int]) -> None:
        self.counts = counts

    def to_dict(self) -> Dict[str, Any]:
        return {"module": "count", "counts": self.counts}

    def render(self, **options: Any) -> str:
        return "\n".join(f"{kind}: {n}" for kind, n in sorted(self.counts.items()))
```

Keep the guarantees

A new module must preserve Fou's contract: ***deterministic ordering, no printing from library methods, dedicated exceptions for bad arguments, and recorded (not raised) errors*** *for problems in the project being inspected.*

16 The Test Suite and a Checklist

Verify, then share — *Fou ships a pytest suite in* `tests/`*, organized by concern:*

File	Covers
`conftest.py`	*shared fixtures (sample projects in temp directories)*
`test_options.py`	*argument validation and error messages*
`test_scanner.py`	*traversal, depth, exclusion, hidden files, ordering*
`test_parser.py`	*extraction of every element kind*
`test_linecount.py`	*code / blank / comment / docstring classification*
`test_results.py`	*InspectionResult, rendering, JSON and saving*
`test_lazy_and_index.py`	*caching, indexing, lazy counting, refresh()*
`test_links.py`	*schemes, templates, OSC 8 output, link modes*
`test_cli.py`	*the command line and its exit codes*

```
$ pip install -e ".[dev]"
$ pytest
```

Run the suite before every release; plain `pytest` *from the project root is enough.*

Before you publish: a checklist

Item	Why it matters	Status
Unique package name	*PyPI names are first-come, first-served*	*verify yourself*
LICENSE file	*pyproject declares MIT, but the text should ship*	***missing: add***
Author and URLs	*who wrote it + link to source*	*missing: see next page*
Classifiers, keywords	*discoverability, Python versions*	*missing: see next page*
README as long description	*rendered on the PyPI page*	*already configured*
Version	*each upload needs a new, never-used version*	*0.1.0 in* `_version.py`
Typing marker	*tells type checkers the package is typed*	*present (* `py.typed` *)*

Check the name first

If `fou` *is taken on PyPI, pick another (e.g.* `fou-inspect`*) via* `name` *in pyproject. The import folder* `fou/` *can stay.*

16 pyproject.toml (1/2): Metadata

The current file, extended with author, URLs, keywords and classifiers. README and version setup are unchanged; replace placeholders as needed.

```
[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"

[project]
name = "fou"
dynamic = ["version"]
description = "Developer-controlled codebase inspection and navigation (not an AI agent)"
readme = "README.md"
requires-python = ">=3.9"
license = { text = "MIT" }
authors = [{ name = "Mohammad Mehdi Masoumi" }]
keywords = ["code inspection", "ast", "static analysis", "codebase navigation", "cli"]
classifiers = [
    "Development Status :: 3 - Alpha",
    "Intended Audience :: Developers",
    "License :: OSI Approved :: MIT License",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.9",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Topic :: Software Development",
    "Typing :: Typed",
]
dependencies = []
```

Reading it

- `dynamic = ["version"]` — *the version is not written here; it is read from* `_version.py` *(chapter 15).*
- `dependencies = []` — *Fou's promise of* ***zero dependencies****, in one line.*
- `requires-python = ">=3.9"` *matches the requirement from chapter 2.*
- ***Classifiers + keywords*** *help people find the package and show supported Python versions.*

16 pyproject.toml (2/2): URLs, Scripts, Tools

```
[project.urls]
Homepage = "https://github.com/mohammadmehdimasoumi/fou"
Source = "https://github.com/mohammadmehdimasoumi/fou"
Issues = "https://github.com/mohammadmehdimasoumi/fou/issues"

[project.optional-dependencies]
dev = ["pytest>=7"]

[project.scripts]
fou = "fou.cli:main"

[tool.setuptools.dynamic]
version = { attr = "fou._version.__version__" }

[tool.setuptools.packages.find]
include = ["fou*"]

[tool.setuptools.package-data]
fou = ["py.typed"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

The three URLs assume a repository named `fou` under your GitHub account — adjust them to the real address.

Section	What it does
`[project.scripts]`	*creates the* `fou` *console command →* `fou.cli:main`
`[project.optional-dependencies]`	*the* `dev` *extra (*`pip install -e ".[dev]"`*)*
`[tool.setuptools.dynamic]`	*reads the version from one place*
`[tool.setuptools.package-data]`	*ships the* `py.typed` *marker*
`[tool.pytest.ini_options]`	*plain* `pytest` *finds* `tests/`

Licence field

Recent setuptools also accept the SPDX style: `license = "MIT"` *together with* `license-files = ["LICENSE"]`*. If your build prints a deprecation warning about the table form, switch.*

16 Build and Rehearse on TestPyPI

Build the distribution

Install the two standard tools once, then build from the project root (the folder with `pyproject.toml`):

```
$ python -m pip install --upgrade build twine
$ python -m build
Successfully built fou-0.1.0.tar.gz and fou-0.1.0-py3-none-any.whl
```

A `dist/` folder appears with a source distribution (`.tar.gz`) and a **wheel** (`.whl`). Check that metadata and README will render on PyPI:

```
$ twine check dist/*
Checking dist/fou-0.1.0-py3-none-any.whl: PASSED
Checking dist/fou-0.1.0.tar.gz: PASSED
```

Rehearse on TestPyPI

TestPyPI is a separate copy of the index for practice, with its own account and tokens. Uploading there first catches mistakes without using up a real version number.

1. Create an account at **test.pypi.org** and enable two-factor authentication.
2. Create an API token in the account settings.
3. Upload with twine: username `__token__`, password = the token.

```
$ twine upload --repository testpypi dist/*
```

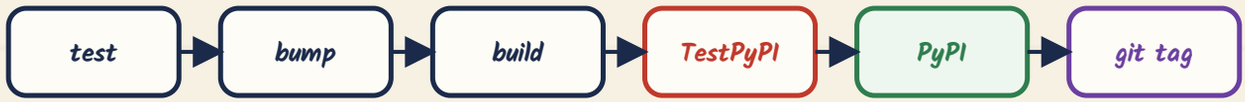
Then install into a clean virtual environment and try it (the extra index lets pip find dependencies on real PyPI — Fou has none, but the habit is good):

```
$ python -m venv /tmp/try && . /tmp/try/bin/activate
$ pip install --index-url https://test.pypi.org/simple/ \
    --extra-index-url https://pypi.org/simple/ fou
$ fou --version
```

16 Publish for Real

```
$ twine upload dist/*
```

After a few moments the package is live at ***pypi.org/project/fou/*** *and anyone can run* `pip install fou`.



the release loop

Things to remember

- ***Versions are permanent.*** *A file name can never be uploaded twice, even if you delete the release. Fix a mistake by publishing* `0.1.1`*, not by re-uploading* `0.1.0`*.*
- ***Bump the version in one place,*** `fou/_version.py`*, then rebuild. Delete old files in* `dist/` *first so you don't upload them again.*
- ***Guard your token.*** *Never commit it. Prefer a token scoped to one project once the first release exists.*

Semantic versioning

Version	When
`0.1.1`	*fixes*
`0.2.0`	*new features*
`1.0.0`	*when the API is stable*

> **Tag every release**
> *Create a git tag such as* `v0.1.0` *for each upload, so users — and you — can always find the exact source that produced a given PyPI version.*

17 Known Limitations and FAQ

Known limitations

- ***Static analysis only.*** *setattr, exec, dynamic classes and decorator-generated names are invisible.*
- ***Python version matters.*** *Source must parse with the Python running Fou; newer syntax on an older interpreter is a parse error for that file.*
- ***Syntax errors yield no elements*** *— just an error entry and an approximate line count.*
- *Only* `.py` *files are analyzed (*`.pyi` *stubs and notebooks are not).*
- *Module names come from* `__init__.py`*; namespace packages without them are named by file only.*
- `location` *must be a directory — to focus on one file, filter by* `element.path`*.*

Frequently asked questions

Why is my class not under functions? *Classes are their own type; a* `def` *directly in a class is a method; async functions are under* `async_functions`*.*

Why does a file show "binary" or a different line count? *Non-Python files can't be split into code and comments: text files report non-blank lines, binary-looking files are labeled binary.*

Why does the code-line count differ from my editor's? *The editor counts physical lines; Fou counts real code only.* `LineStats` *has every number.*

A decorated function starts on the decorator line — a bug? *No. The span starts at the first decorator;* `location.lineno` *still points at the* `def`*, and* `details["start_lineno"]` *gives the decorator start.*

I edited a file but results didn't change. *Results are cached: call* `project.refresh()` *or create a new project.*

Does Fou follow symbolic links? *No — never, which avoids loops and keeps results predictable.*

17 Troubleshooting

Links

Symptom	Likely cause and fix
No underline, nothing to click	output is not an interactive terminal, or no OSC 8 support. Try `--links always` or a supported terminal.
Click opens the file, not the line	you are on the default `file` scheme. Use `--editor vscode` (or your editor's scheme).
Clicking does nothing	try Ctrl+click / Cmd+click; check that your editor registered its URL handler.
Unknown link scheme error	a typo — the message suggests the closest scheme.
Links vanished inside tmux	tmux needs version 3.4 or newer for OSC 8.
macOS Terminal.app	no OSC 8 support. Use iTerm2 or the VS Code terminal.

Installation

Symptom	Likely cause and fix
`fou: command not found`	scripts folder not on PATH. Use `python -m fou ...`.
No matching distribution found	not published yet, or the name differs. Install from source: `pip install -e .`.
SyntaxError for a file in your project	it is reported in `result.errors`. Fix the file, or exclude it.
Output shows too many files	add exclusions or lower `depth`. A custom exclude **replaces** the defaults.

> **Rule of thumb**
> Links not working? Check **terminal support**, then the **scheme**, then the **modifier key**.

A Cheat Sheet — Everything on One Page

Python API

```
from fou import inspect
project = inspect(location, depth="all", extensions=None, exclude=None, hidden=False)

project.files(show_lines=True)      project.classes(details=False)
project.imports(details=False)      project.methods(details=False)
project.functions(details=False)    project.variables(details=False)
project.async_functions(...)        project.constants(details=False)
project.decorators(details=False)   project.refresh()
project.query(type="all", details=False, show_lines=True)

result.show(links="auto", scheme=None)    result.render(links=False, scheme=None)
result.to_dict()   result.to_json()       result.save(format="json", path="out.json")
result["classes"]  result.errors  result.types  len(result)  for el in result: ...
```

Command line

```
fou inspect [location] [--depth N] [-e EXT]... [-x GLOB]... [--hidden]
            [-t TYPE[,TYPE]...] [-d] [--no-lines] [--json]
            [--links auto|always|never] [--editor SCHEME] [-o out.json]
python -m fou inspect ...
export FOU_LINK_SCHEME=vscode        export FOU_LINKS=always|never
```

Types, schemes and exceptions

Inspection types	*files, imports, functions, async_functions, classes, methods, variables, constants, decorators*
Link schemes	*file, vscode, vscode-insiders, cursor, pycharm, idea, sublime, textmate, or a template*
Template placeholders	`{path} {fspath} {uri} {line} {col}`
Variable scopes	*module, class, instance, local*
Method types	*instance, class, static, property*
Exceptions	*FouError > LocationError, InvalidOptionError (> Invalid*Error), SaveError*
Exit codes	*0 success, 2 error*

B Glossary — Terms Used in This Notebook

Term	Meaning
AST	Abstract Syntax Tree: the structured form of Python source produced by the standard `ast` module.
Atomic write	Writing a file so it is either fully written or not changed at all.
Code line	A line holding real code, excluding blanks, comment-only lines and docstrings.
Deterministic	Producing exactly the same output every time for the same input.
Docstring	A string literal at the start of a module, class or function that documents it.
Element	One discovered code item: an import, function, method, class, variable, constant or decorator.
Glob	A shell-style pattern with wildcards such as `*` and `?`.
Lazy evaluation	Doing work only when its result is first needed, then caching it.
OSC 8	A terminal escape sequence that turns text into a clickable hyperlink.
Qualified name	A dotted name including enclosing scopes, such as `Cart.total`.
Scheme	A URL template deciding which program opens a link and whether it jumps to a line.
Static analysis	Examining code without running it.
Tokenize	Python's standard module that splits source into tokens, used here to classify lines.
Wheel	A built, ready-to-install Python package file (`.whl`).

About the Author

Mohammad Mehdi Masoumi *is the author of Fou and of the complete guide this notebook is based on.*

Mohammad Mehdi Masoumi

github.com/mohammadmehdimasoumi

author of Fou · Python · static analysis

Fou is released under the MIT license, as declared in the project's `pyproject.toml`*. If this notebook helped you, if you found a mistake, or if you have an idea for the next Fou module, the best place to get in touch is GitHub:*

`github.com/mohammadmehdimasoumi`

The whole notebook in five lines

1. ***inspect(path)*** *returns a Project — instantly, nothing is scanned yet.*
2. *Ask a question with* ***classes()****,* ***functions()****,* ***imports()****…* *or* ***query(type=[…])****.*
3. *Add* ***details=True*** *for signatures, docstrings, decorators and code-line counts.*
4. *Print it, loop over it,* ***save it as JSON****, or* ***click it*** *in the terminal.*
5. *Problems in the project are* ***recorded*** *in* `result.errors`*; mistakes in your arguments* ***raise****.*

Thank you for reading

Fou is deterministic, offline and fully under your control. Happy inspecting! ✏️

— Mohammad Mehdi Masoumi · github.com/mohammadmehdimasoumi